

P1644R0

Add wait/notify to `atomic<shared_ptr<T>>`

Published Proposal, 2019-06-15

Author:

[David Olsen](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

SG1, LEWG, LWG

Table of Contents

1 Introduction

2 Changelog

3 Wording

References

Informative References

§ 1. Introduction

[\[P1135\]](#) added the member functions `wait`, `notify_one`, and `notify_all` to `atomic<T>`, but did not add those same member functions to `atomic<shared_ptr<T>>` or `atomic<weak_ptr<T>>` due to an oversight. This paper takes care of that, bringing the interfaces of `atomic<shared_ptr<T>>` and `atomic<weak_ptr<T>>` back in line with that of `atomic<T>`.

§ 2. Changelog

Revision 0: Initial version.

§ 3. Wording

Note: The following changes are relative to the post Kona 2019 working draft of ISO/IEC 14882, ([N4810]), with the changes from [P1135R5] merged in.

Modify the class synopsis in [\[util.smartptr.atomic.shared\]](#) as follows:

```
namespace std {
    template<class T> struct atomic<shared_ptr<T>> {

        // ...

        bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T> desired,
                                     memory_order order = memory_order::seq_cst) noexcept;

        void wait(shared_ptr<T> old, memory_order order = memory_order::seq_cst) noexcept;
        void notify_one() noexcept;
        void notify_all() noexcept;

        constexpr atomic() noexcept = default;
        // ...
    };
}
```

Add the following to the end of [\[util.smartptr.atomic.shared\]](#):

```
void wait(shared_ptr<T> old, memory_order order = memory_order::seq_cst) {
```

1 *Expects:* order is neither `memory_order::release` nor `memory_order::acq_rel`.

2 *Effects:* Repeatedly performs the following steps, in order:

(2.1) — Evaluates `load(order)` and compares it to `old`.

(2.2) — If the two are not equivalent, returns.

(2.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

3 *Remarks:* Two `shared_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation ([**atomics.wait**]).

```
void notify_one() noexcept;
```

4 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked ([**atomics.wait**]) by this call, if any such atomic waiting operations exist.

5 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

```
void notify_all() noexcept;
```

6 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked ([**atomics.wait**]) by this call.

7 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

Modify the class synopsis in [\[util.smartptr.atomic.weak\]](#) as follows:

```

namespace std {
    template<class T> struct atomic<weak_ptr<T>> {

        // ...

        bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T> desired,
                                     memory_order order = memory_order::seq_cst) noexcept;

        void wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst) noexcept;
        void notify_one() noexcept;
        void notify_all() noexcept;

        constexpr atomic() noexcept = default;
        // ...
    };
}

```

Add the following to the end of [\[util.smartptr.atomic.weak\]](#):

```
void wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst) cor
```

1 *Expects:* order is neither `memory_order::release` nor `memory_order::acq_rel`.

2 *Effects:* Repeatedly performs the following steps, in order:

(2.1) — Evaluates `load(order)` and compares it to `old`.

(2.2) — If the two are not equivalent, returns.

(2.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

3 **Remarks:* Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation ([**atomics.wait**]).

```
void notify_one() noexcept;
```

4 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked ([**atomics.wait**]) by this call, if any such atomic waiting operations exist.

5 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

```
void notify_all() noexcept;
```

6 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked ([**atomics.wait**]) by this call.

7 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

§ References

§ Informative References

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4810). 15 March 2019. URL: <https://wg21.link/n4810>